Task 1 :

Social media platforms generate huge amounts of text data every day. Sentiment analysis helps identify whether users express positive, negative, or neutral opinions in their posts. In this activity, I used a Twitter dataset containing around 1.6 million tweets and performed sentiment analysis using two different approaches: a VADER sentiment analyzer and a lexicon-based sentiment analysis model. Hadoop MapReduce was used to process the dataset and generate sentiment statistics and evaluation metrics.

Task 2 :

This activity mentions collecting data using Twitter or Reddit APIs. Instead of collecting new data, I used the publicly available Sentiment140 dataset because it already contains a large number of labelled tweets suitable for sentiment analysis.

Uploading dataset to HDFS

```
$ hdfs dfs -mkdir -p /twitter/amisha
$ hdfs dfs -put /home/hduser/Downloads/twitter_Amisha.csv /twitter/amisha/
```

Preprocessing : Before going to the sentiment analysis, i created one preprocessed_dataset.py by doing simple data cleaning and pre-processing on the original data, which did :

Preprocessing Step	Purpose
Lowercase Conversion	Normalize text
URL Removal	Remove web links
Mention Removal	Remove Twitter usernames
Hashtag Processing	Keep hashtag words
Whitespace Cleanup	Standardize text
Missing Tweet Removal	Improve data quality
Invalid Label Removal	Keep valid sentiment labels
Malformed Row Removal	Prevent processing errors

$ nano preprocess_dataset.py

```
import csv
import re

INPUT_FILE = "/home/hduser/Downloads/twitter_Amisha.csv"

OUTPUT_FILE = "/home/hduser/Downloads/twitter_preprocessed.csv"

def clean_tweet(tweet):

    # Convert to lowercase
    tweet = tweet.lower()

    # Remove URLs
    tweet = re.sub(r"http\S+|www\S+", "", tweet)

    # Remove Twitter mentions
    tweet = re.sub(r"@\w+", "", tweet)

    # Remove hashtag symbol but preserve the word
    tweet = tweet.replace("#", "")

    # Remove extra spaces
    tweet = " ".join(tweet.split())

    return tweet

total_rows = 0
processed_rows = 0
missing_tweets = 0
invalid_labels = 0
malformed_rows = 0

with open(INPUT_FILE, "r", encoding="utf-8-sig",
          errors="replace", newline="") as infile:

    reader = csv.reader(infile)

    with open(OUTPUT_FILE, "w", encoding="utf-8",
              newline="") as outfile:

        writer = csv.writer(outfile)

        for row in reader:

            total_rows += 1

            # Check required columns
            if len(row) <= 5:
                malformed_rows += 1
                continue

            # Extract sentiment label
            label = row[0].strip()
```

```
            total_rows += 1

            # Check required columns
            if len(row) <= 5:
                malformed_rows += 1
                continue

            # Extract sentiment label
            label = row[0].strip()

            # Keep only valid labels
            if label not in ["0", "4"]:
                invalid_labels += 1
                continue

            # Extract tweet
            tweet = row[5].strip()

            # Remove missing tweets
            if not tweet or tweet.lower() in ["nan", "null", "none"]:
                missing_tweets += 1
                continue

            # Apply basic preprocessing
            cleaned_tweet = clean_tweet(tweet)

            # Remove empty tweets after cleaning
            if not cleaned_tweet:
                missing_tweets += 1
                continue

            # Replace original tweet with cleaned tweet
            row[5] = cleaned_tweet

            # Save the complete CSV row
            writer.writerow(row)

            processed_rows += 1

print("\n========== DATA PREPROCESSING REPORT ==========")

print("Original Dataset Rows:", total_rows)

print("Successfully Processed Rows:", processed_rows)

print("Missing Tweets Removed:", missing_tweets)

print("Invalid Labels Removed:", invalid_labels)

print("Malformed Rows Removed:", malformed_rows)
```

run the file
$ python3 preprocess_dataset.py

```
hduser@pc2:~$ python3 preprocess_dataset.py

========== DATA PREPROCESSING REPORT ==========
Original Dataset Rows: 1600000
Successfully Processed Rows: 1597185
Missing Tweets Removed: 2815
Invalid Labels Removed: 0
Malformed Rows Removed: 0
hduser@pc2:~$
```

and after executing the preprocess_dataset.py, we will get twitter_preprocessed.csv file in the Downloads folder because of using this command :

OUTPUT_FILE = "/home/hduser/Downloads/twitter_preprocessed.csv"
with open(OUTPUT_FILE, "w", encoding="utf-8", newline="") as outfile:
If file doesn't exist, it created by it's own because it opened by "w", means "write mode". copy

twitter_preprocessed.csv dataset to HDFS :
$ hdfs dfs -put /home/hduser/Downloads/twitter_preprocessed.csv /twitter/amisha/

```
hduser@pc2:~$ hdfs dfs -ls /twitter/amisha/
Found 1 items
-rw-r--r--   2 hduser supergroup  238942690 2026-11-01 13:17 /twitter/amisha/twitter_Amisha.csv
hduser@pc2:~$ hdfs dfs -put /home/hduser/Downloads/twitter_preprocessed.csv /twitter/amisha/
hduser@pc2:~$ hdfs dfs -ls /twitter/amisha/
Found 2 items
-rw-r--r--   2 hduser supergroup  238942690 2026-11-01 13:17 /twitter/amisha/twitter_Amisha.csv
-rw-r--r--   2 hduser supergroup  208413488 2026-11-04 16:00 /twitter/amisha/twitter_preprocessed.csv
```

Task 3 : Sentiment Analysis

For this activity, I experimented with two different sentiment analysis approaches:

1. VADER (Valence Aware Dictionary and Sentiment Reasoner)
2. Lexicon-Based Sentiment Analysis

Model 1: VADER Sentiment Analyzer

For the first experiment, I used the VADER (Valence Aware Dictionary and Sentiment Reasoner) sentiment analyzer. VADER is designed specifically for social media text and can handle informal language, punctuation, emojis, and common internet expressions.

The mapper reads each tweet from the dataset, extracts the sentiment label and tweet text, and then applies VADER sentiment analysis.

The VADER model generates a compound score between -1 and +1.

Classification was performed using the default VADER thresholds:

compound >= 0.05 → Positive
compound <= -0.05 → Negative
otherwise → Neutral

Mapper :

$ nano twitter_mapper.py

```python
import sys
import csv
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

for row in csv.reader(sys.stdin):

    try:
        # Extract CSV columns
        original_label = row[0].strip()
        tweet = row[5].strip()

        if not tweet:
            continue

        # Convert original dataset labels
        if original_label == "0":
            actual_sentiment = "Negative"
        elif original_label == "4":
            actual_sentiment = "Positive"
        else:
            continue

        # VADER sentiment analysis
        scores = analyzer.polarity_scores(tweet)

        compound = scores['compound']

        # Classify sentiment
        if compound >= 0.05:
            predicted_sentiment = "Positive"

        elif compound <= -0.05:
            predicted_sentiment = "Negative"

        else:
            predicted_sentiment = "Neutral"

        # Emit key-value pair
        print(
            f"SENTIMENT\t"
            f"{predicted_sentiment}|"
            f"{actual_sentiment}|"
            f"{compound}"
        )

    except (IndexError, ValueError):
        continue
```

Model 2: Lexicon-Based Sentiment Analysis

For the second experiment, I implemented a simple lexicon-based approach using predefined positive and negative word lists. The mapper counts the occurrence of positive and negative words in each tweet and calculates a sentiment score.

Based on the final score, tweets are classified as Positive, Negative, or Neutral.

$ nano twitter_mapper.py

```
import sys
import csv

POSITIVE_WORDS = {"good", "great", "awesome", "love", "happy", "best", "nice", "wonderful", "positive", "amazing", "thanks", "cool", "exciting"}
NEGATIVE_WORDS = {"bad", "awful", "terrible", "hate", "sad", "worst", "poor", "horrible", "negative", "angry", "disappoint", "useless", "fail", "wrong", "boring"}

for row in csv.reader(sys.stdin):
    try:
        label = row[0].strip()
        tweet = row[5].strip()

        if not tweet:
            continue

        if label == "0":
            actual = "Negative"
        elif label == "4":
            actual = "Positive"
        else:
            continue

        # Count positive and negative words in the tweet
        words = set(tweet.lower().split())
        pos_count = len(words.intersection(POSITIVE_WORDS))
        neg_count = len(words.intersection(NEGATIVE_WORDS))

        # Calculate simple compound score (-1.0 to 1.0)
        total = pos_count + neg_count
        if total == 0:
            score = 0.0
        else:
            score = (pos_count - neg_count) / total

        # Classify sentiment
        if score > 0:
            predicted = "Positive"
        elif score < 0:
            predicted = "Negative"
        else:
            predicted = "Neutral"

        print(f"SENTIMENT\t{predicted}|{actual}|{score}")

    except (IndexError, ValueError):
        continue
```

Reducer

After the mapper generates sentiment predictions, the reducer collects all mapper outputs and performs the final analysis.
The reducer receives records : SENTIMENT Positive|Negative|0.7345

where:

- Positive = Predicted Sentiment
- Negative = Actual Sentiment
- 0.7345 = Compound Score

Instead of analyzing individual tweets, the reducer aggregates the results generated by all mapper tasks and produces final evaluation metrics.

The reducer:

- Counts the number of Positive, Negative, and Neutral tweets.
- Calculates the average compound score for each sentiment category.
- Builds a confusion matrix using actual and predicted sentiments.
- Calculates evaluation metrics such as Accuracy, Precision, Recall, and F1-Score.
- Generates the final sentiment analysis report.

$ nano twitter_reducer.py

```
import sys

sentiment_count = {
    "Positive": 0,
    "Negative": 0,
    "Neutral": 0
}
sentiment_sum = {
    "Positive": 0.0,
    "Negative": 0.0,
    "Neutral": 0.0
}
confusion = {
    "Positive": {"Positive": 0, "Negative": 0, "Neutral": 0},
    "Negative": {"Positive": 0, "Negative": 0, "Neutral": 0}
}
total = 0

for line in sys.stdin:
    try:
        key, value = line.strip().split('\t', 1)

        if key != "SENTIMENT":
            continue

        predicted, actual, score = value.split('|')
        score = float(score)
        sentiment_count[predicted] += 1
        sentiment_sum[predicted] += score
        confusion[actual][predicted] += 1
        total += 1

    except (ValueError, KeyError):
        continue

print("\n========== SENTIMENT DISTRIBUTION ==========")

for sentiment in ["Positive", "Negative", "Neutral"]:
    count = sentiment_count[sentiment]
    average = sentiment_sum[sentiment] / count if count else 0
    percentage = (count / total * 100) if total else 0
    print(f"{sentiment}: {count}")
```

```
    print(f"Percentage: {percentage:.2f}%")
    print(f"Average Compound Score: {average:.4f}")

print("--------------------------------")
print("\n========== MODEL EVALUATION ==========")

correct = (confusion["Positive"]["Positive"]+ confusion["Negative"]["Negative"])
accuracy = correct / total if total else 0
print(f"Total Tweets: {total}")
print(f"Correct Predictions: {correct}")
print(f"Accuracy: {accuracy * 100:.2f}%")

for sentiment in ["Positive", "Negative"]:
    tp = confusion[sentiment][sentiment]
    fp = sum(
        confusion[actual][sentiment]
        for actual in ["Positive", "Negative"]
        if actual != sentiment
    )
    fn = sum(
        confusion[sentiment][predicted]
        for predicted in ["Positive", "Negative", "Neutral"]
        if predicted != sentiment
    )
    precision = tp / (tp + fp) if (tp + fp) else 0
    recall = tp / (tp + fn) if (tp + fn) else 0
    f1 = (
        2 * precision * recall / (precision + recall)
        if (precision + recall) else 0
    )

print(f"\n{sentiment} Metrics")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
print("\n========== CONFUSION MATRIX ==========")
print("Actual / Predicted")
print("Positive Negative Neutral")
for actual in ["Positive", "Negative"]:
    print(
        f"{actual:12}"
        f"{confusion[actual]['Positive']:8}"
        f"{confusion[actual]['Negative']:8}"
        f"{confusion[actual]['Neutral']:8}"
    )
```

After creating the mapper and reducer programs, Hadoop Streaming was used to execute the sentiment analysis pipeline on the Twitter dataset stored in HDFS.

$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming*.jar -mapper "python3 twitter_mapper.py" -reducer "python3 twitter_reducer.py" -input /twitter/amisha/twitter_Amisha.csv -output /twitter/amisha/output_original

for processed data

$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming*.jar -mapper "python3 twitter_mapper.py" -reducer "python3 twitter_reducer.py" -input /twitter/amisha/twitter_preprocessed.csv -output /twitter/amisha/output_preprocessed

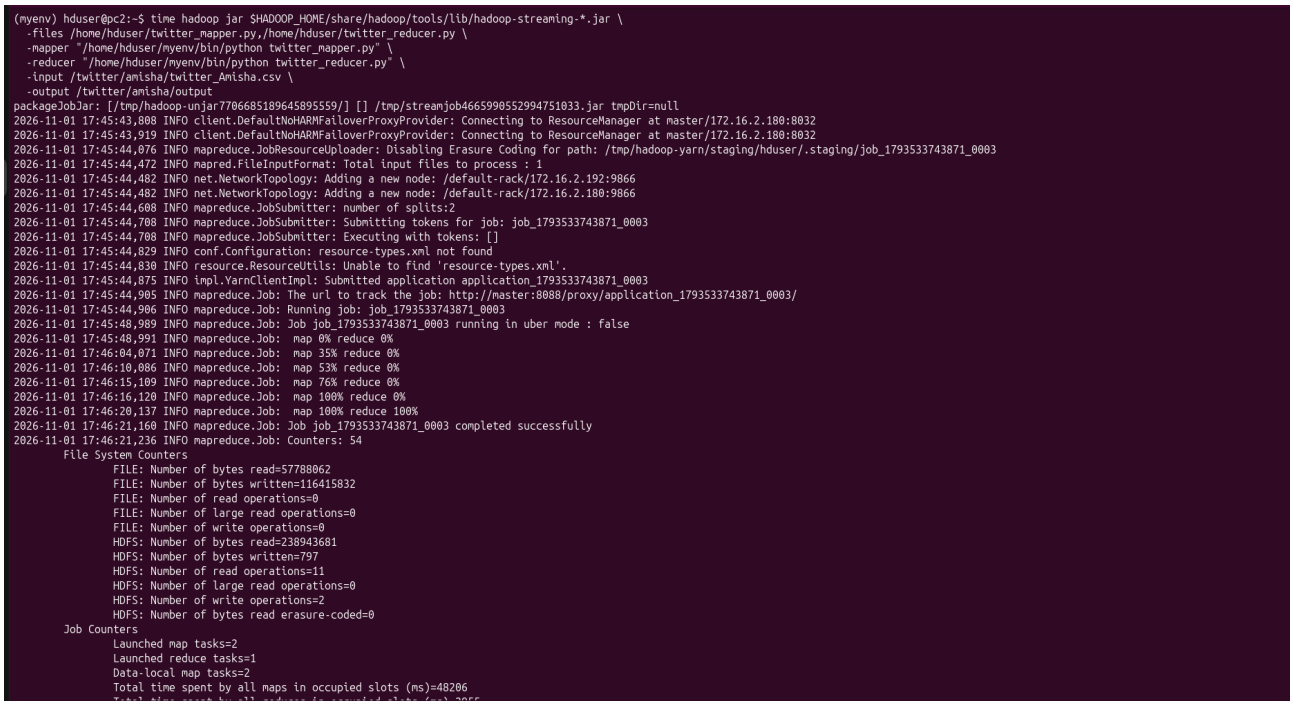
The mapper analyzed each tweet and generated sentiment predictions. Hadoop automatically performed the shuffle and sort phase and passed the intermediate results to the reducer. The reducer then generated sentiment distribution statistics and evaluation metrics

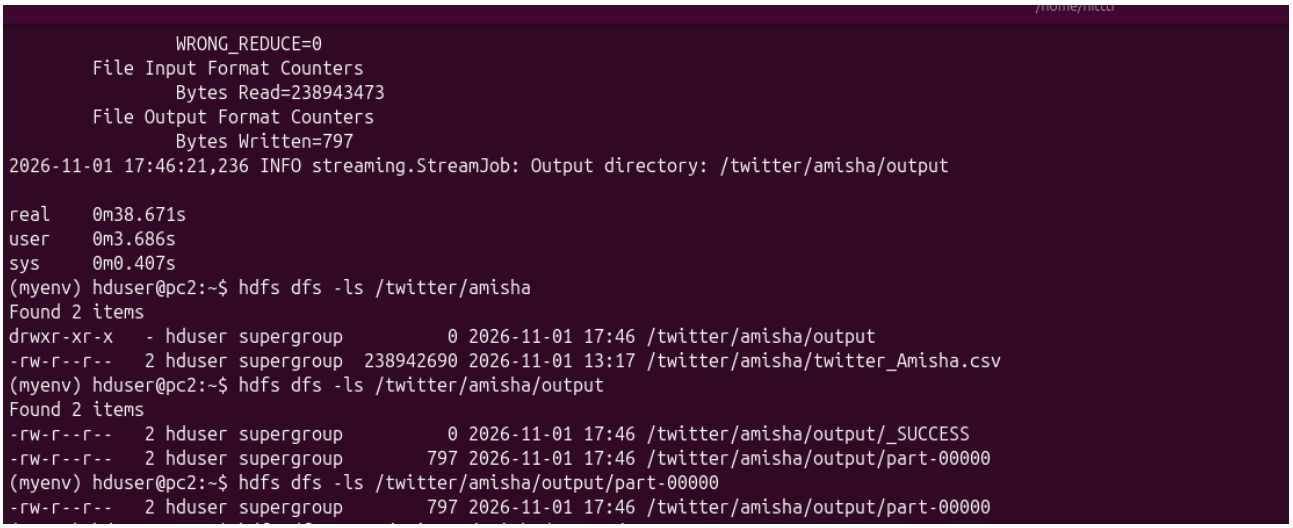
Lexicon-Based Sentiment Analysis

```
hduser@pc2:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files twitter_mapper.py,twitter_reducer.py -mapper "python3 twitter_mapper.py" -reducer "python
3 twitter_reducer.py" -input /twitter/amisha/twitter_Amisha.csv -output /twitter/amisha/output_original
packageJobJar: [/tmp/hadoop-unjar34107452396465970 7/] [] /tmp/streamjob5106116533283583221.jar tmpDir=null
2026-11-04 16:57:04,533 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at master/172.16.2.180:8032
2026-11-04 16:57:04,682 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at master/172.16.2.180:8032
2026-11-04 16:57:04,856 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hduser/.staging/job_1793790590769_0007
2026-11-04 16:57:05,230 INFO mapred.FileInputFormat: Total input files to process : 1
2026-11-04 16:57:05,241 INFO net.NetworkTopology: Adding a new node: /default-rack/172.16.2.192:9866
2026-11-04 16:57:05,241 INFO net.NetworkTopology: Adding a new node: /default-rack/172.16.2.235:9866
2026-11-04 16:57:05,335 INFO mapreduce.JobSubmitter: number of splits:2
2026-11-04 16:57:05,419 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1793790590769_0007
2026-11-04 16:57:05,419 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-11-04 16:57:05,538 INFO conf.Configuration: resource-types.xml not found
2026-11-04 16:57:05,538 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-11-04 16:57:05,586 INFO impl.YarnClientImpl: Submitted application application_1793790590769_0007
2026-11-04 16:57:05,616 INFO mapreduce.Job: The url to track the job: http://master:8088/proxy/application_1793790590769_0007/
2026-11-04 16:57:05,617 INFO mapreduce.Job: Running job: job_1793790590769_0007
2026-11-04 16:57:09,730 INFO mapreduce.Job: Job job_1793790590769_0007 running in uber mode : false
2026-11-04 16:57:09,732 INFO mapreduce.Job:  map 0% reduce 0%
2026-11-04 16:57:15,838 INFO mapreduce.Job:  map 100% reduce 0%
2026-11-04 16:57:21,919 INFO mapreduce.Job:  map 100% reduce 100%
2026-11-04 16:57:21,937 INFO mapreduce.Job: Job job_1793790590769_0007 completed successfully
2026-11-04 16:57:22,030 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=53208530
                FILE: Number of bytes written=107256603
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=238943681
                HDFS: Number of bytes written=796
                HDFS: Number of read operations=11
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
        Job Counters
                Launched map tasks=2
                Launched reduce tasks=1
                Data-local map tasks=2
                Total time spent by all maps in occupied slots (ms)=8701
                Total time spent by all reduces in occupied slots (ms)=3085
                Total time spent by all map tasks (ms)=8701
                Total time spent by all reduce tasks (ms)=3085
                Total vcore-milliseconds taken by all map tasks=8701
                Total vcore-milliseconds taken by all reduce tasks=3085
```

```
hduser@pc2:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files twitter_mapper.py,twitter_reducer.py -mapper "python3 twitter_mapper.py" -reducer "python3 twitter_reducer.py
" -input /twitter/amisha/twitter_preprocessed.csv -output /twitter/amisha/output_preprocessed
packageJobJar: [/tmp/hadoop-unjar4466383762564041696/] [] /tmp/streamjob1119230736934143689.jar tmpDir=null
2026-11-04 17:01:28,301 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at master/172.16.2.180:8032
2026-11-04 17:01:28,411 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at master/172.16.2.180:8032
2026-11-04 17:01:28,570 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hduser/.staging/job_1793790590769_0008
2026-11-04 17:01:28,995 INFO mapred.FileInputFormat: Total input files to process : 1
2026-11-04 17:01:29,016 INFO net.NetworkTopology: Adding a new node: /default-rack/172.16.2.192:9866
2026-11-04 17:01:29,016 INFO net.NetworkTopology: Adding a new node: /default-rack/172.16.2.180:9866
2026-11-04 17:01:29,127 INFO mapreduce.JobSubmitter: number of splits:2
2026-11-04 17:01:29,226 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1793790590769_0008
2026-11-04 17:01:29,226 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-11-04 17:01:29,351 INFO conf.Configuration: resource-types.xml not found
2026-11-04 17:01:29,351 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-11-04 17:01:29,397 INFO impl.YarnClientImpl: Submitted application application_1793790590769_0008
2026-11-04 17:01:29,426 INFO mapreduce.Job: The url to track the job: http://master:8088/proxy/application_1793790590769_0008/
2026-11-04 17:01:29,427 INFO mapreduce.Job: Running job: job_1793790590769_0008
2026-11-04 17:01:33,546 INFO mapreduce.Job: Job job_1793790590769_0008 running in uber mode : false
2026-11-04 17:01:33,548 INFO mapreduce.Job:  map 0% reduce 0%
2026-11-04 17:01:39,664 INFO mapreduce.Job:  map 100% reduce 0%
2026-11-04 17:01:45,739 INFO mapreduce.Job:  map 100% reduce 100%
2026-11-04 17:01:45,762 INFO mapreduce.Job: Job job_1793790590769_0008 completed successfully
2026-11-04 17:01:45,854 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=53116339
                FILE: Number of bytes written=107072251
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=208417804
                HDFS: Number of bytes written=796
                HDFS: Number of read operations=11
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
        Job Counters
                Launched map tasks=2
                Launched reduce tasks=1
                Data-local map tasks=2
                Total time spent by all maps in occupied slots (ms)=8261
                Total time spent by all reduces in occupied slots (ms)=2691
                Total time spent by all map tasks (ms)=8261
                Total time spent by all reduce tasks (ms)=2691
                Total vcore-milliseconds taken by all map tasks=8261
                Total vcore-milliseconds taken by all reduce tasks=2691
                Total megabyte-milliseconds taken by all map tasks=8459264
                Total megabyte-milliseconds taken by all reduce tasks=2755584
```

VADER Sentiment Analyzer





After running the Hadoop MapReduce job on the original dataset,, checked the HDFS directory using hdfs dfs -ls. The _SUCCESS file confirms that the job completed successfully, and part-00000 contains the final output generated by the reducer.

For Vedar sentiment analysis :

$ hdfs dfs -ls /twitter/amisha

$ hdfs dfs -ls /twitter/amisha/output

$ hdfs dfs -cat /twitter/amisha/output/part-00000

```
(myenv) hduser@pc2:~$ hdfs dfs -cat /twitter/amisha/output/part-00000

========== SENTIMENT DISTRIBUTION ==========
Positive: 750276
Percentage: 46.89%
Average Compound Score: 0.5582
------------------------------
Negative: 417690
Percentage: 26.11%
Average Compound Score: -0.4499
------------------------------
Neutral: 432034
Percentage: 27.00%
Average Compound Score: 0.0001
------------------------------

========== MODEL EVALUATION ==========
Total Tweets: 1600000
Correct Predictions: 838703
Accuracy: 52.42%

Positive Metrics
Precision: 0.6649
Recall: 0.6236
F1 Score: 0.6436

Negative Metrics
Precision: 0.8136
Recall: 0.4248
F1 Score: 0.5582

========== CONFUSION MATRIX ==========
Actual / Predicted
              Positive Negative Neutral
Positive      498863   77850  223287
Negative      251413  339840  208747
```

For Lexicon-Based Sentiment Analysis :

original :

$ hdfs dfs -ls /twitter/amisha

$ hdfs dfs -ls /twitter/amisha/output_original

$ hdfs dfs -cat /twitter/amisha/output_original/part-00000

```
hduser@pc2:~$ hdfs dfs -ls /twitter/amisha/output_original
Found 2 items
-rw-r--r--   2 hduser supergroup          0 2026-11-04 16:57 /twitter/amisha/output_original/_SUCCESS
-rw-r--r--   2 hduser supergroup        796 2026-11-04 16:57 /twitter/amisha/output_original/part-00000
hduser@pc2:~$ hdfs dfs -cat /twitter/amisha/output_original/part-00000

========== SENTIMENT DISTRIBUTION ==========
Positive: 244891
Percentage: 15.31%
Average Compound Score: 0.9987
------------------------------
Negative: 76649
Percentage: 4.79%
Average Compound Score: -0.9981
------------------------------
Neutral: 1278460
Percentage: 79.90%
Average Compound Score: 0.0000
------------------------------

========== MODEL EVALUATION ==========
Total Tweets: 1600000
Correct Predictions: 251299
Accuracy: 15.71%

Positive Metrics
Precision: 0.7506
Recall: 0.2298
F1 Score: 0.3518

Negative Metrics
Precision: 0.8806
Recall: 0.0844
F1 Score: 0.1540

========== CONFUSION MATRIX ==========
Actual / Predicted
             Positive Negative Neutral
Positive      183803    9153  607044
Negative       61088   67496  671416
hduser@pc2:~$
```

For preprocessed data :

$ hdfs dfs -ls /twitter/amisha

$ hdfs dfs -ls /twitter/amisha/output_preprocessed

$ hdfs dfs -cat /twitter/amisha/output_preprocessed/part-00000

```
hduser@pc2:~$ hdfs dfs -ls /twitter/amisha/output_preprocessed
Found 2 items
-rw-r--r--   2 hduser supergroup          0 2026-11-04 17:01 /twitter/amisha/output_preprocessed/_SUCCESS
-rw-r--r--   2 hduser supergroup        796 2026-11-04 17:01 /twitter/amisha/output_preprocessed/part-00000
hduser@pc2:~$ hdfs dfs -cat /twitter/amisha/output_preprocessed/part-00000

========== SENTIMENT DISTRIBUTION ==========
Positive: 244925
Percentage: 15.33%
Average Compound Score: 0.9987
------------------------------
Negative: 76969
Percentage: 4.82%
Average Compound Score: -0.9981
------------------------------
Neutral: 1275291
Percentage: 79.85%
Average Compound Score: 0.0000
------------------------------

========== MODEL EVALUATION ==========
Total Tweets: 1597185
Correct Predictions: 251632
Accuracy: 15.75%

Positive Metrics
Precision: 0.7506
Recall: 0.2302
F1 Score: 0.3523

Negative Metrics
Precision: 0.8808
Recall: 0.0849
F1 Score: 0.1549

========== CONFUSION MATRIX ==========
Actual / Predicted
             Positive Negative Neutral
Positive      183841    9178  605669
Negative       61084   67791  669622
hduser@pc2:~$
```

Comparison of Models

Two sentiment analysis approaches were tested in this activity:

- VADER Sentiment Analyzer
- Lexicon-Based Sentiment Analysis

Metric	VADER	Lexicon
Accuracy	52.42	15.71
Positive F1 Score	0.6436	0.3518
Negative F1 Score	0.5582	0.1540

From the results, VADER performed better than the lexicon-based model. The VADER model achieved higher accuracy and F1-scores because it is designed for social media text and considers sentiment intensity instead of only matching predefined words.

Impact of Preprocessing

The lexicon-based model was tested on both the original dataset and the preprocessed dataset.

Metric	Original Dataset	Preprocessed Dataset
Accuracy	15.71%	15.75%
Positive F1 Score	0.3518	0.3523
Negative F1 Score	0.1540	0.1549

After preprocessing, a small improvement was observed in accuracy and F1-scores. The preprocessing steps removed noise from the dataset and slightly improved the quality of the input data.

Task 4 :

For this activity, Hadoop MapReduce through Hadoop Streaming was used to process the Twitter dataset. The dataset contained around 1.6 million tweets, so it was suitable for large-scale analysis.

The mapper analyzed each tweet and generated sentiment predictions, while the reducer collected the intermediate results and generated the final sentiment statistics and evaluation metrics.

The dataset was stored in HDFS and processed using Hadoop Streaming. After execution, the final output was generated in HDFS as part-00000, which contained sentiment distribution, accuracy, precision, recall, F1-score and the confusion matrix.

This helped process a large dataset and generate results more efficiently compared to processing everything manually on a single file.

Task 5: Insights, Interpretation and Ethical Considerations

Insights and Interpretation

From the results, it was clear that the choice of sentiment analysis method had a big impact on the final output. The VADER model achieved an accuracy of 52.42%, while the lexicon-based model achieved 15.71% accuracy. This shows that more advanced sentiment analysis methods are able to understand social media text better than simple word matching approaches.
After preprocessing, the lexicon-based model showed a small improvement. The accuracy increased from 15.71% to 15.75%, which indicates that cleaning the dataset helped improve the quality of the input data.

The output also showed that a large number of tweets were classified as Neutral in the lexicon-based approach. This affected the overall accuracy because fewer tweets were classified as Positive or Negative.

Ethical Considerations

Social media data contains opinions and content shared by users, so it should be handled responsibly.

Different sentiment analysis models can also produce different results for the same dataset. This means that conclusions can change depending on the method being used.

Models may also struggle with sarcasm, slang, abbreviations, or context. Because of this, sentiment analysis results should be treated as indicators rather than completely accurate representations of user opinion.